INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY, BHUBANESWAR

Department of Computer Science & Engineering | DAA Laboratory (CS3101)

LAB 09 REPORT: ADVANCED GREEDY DESIGN PARADIGM

Student: Rishav Shaw **Roll:** B125100 **Branch:** B.Tech CSE (B1) **Instructor:** Dr. A. K. Dash **Date:** Oct 2026

Executive Summary & Paradigm Foundations: This laboratory investigates the **Greedy Design Paradigm** across ten non-trivial optimization problems. Unlike Dynamic Programming, greedy algorithms make a locally optimal choice at each step without backtracking. Correctness requires proving the **Greedy-Choice Property** (a globally optimal solution can be reached through local decisions) and **Optimal Substructure** via formal mathematical exchange arguments, matroid theory (Rado-Edmonds), and priority-queue invariants.

Index of Lab 09 Problems & Complexity Summary

Q. No.	Problem Name	Algorithmic Paradigm	Time	Space	Status
Q1	Fractional Knapsack (Decay)	Deteriorating Resource Greedy	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q2	Canonical Huffman Coding	Min-Heap Prefix Tree & Bitwise Ordering	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q3	Minimum Refueling Stops	Retroactive Max-Heap (Reverse Greedy)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q4	Minimum Cost Sticks	Min-Heap Huffman Merge Tree	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q5	Candy Distribution	Bi-directional Slope Greedy Scan	$\Theta(n)$	$\Theta(n)$	Verified
Q6	Reorganise String K-Apart	Max-Heap + FIFO Cooldown Queue	$\mathcal{O}(N \log\lvert\Sigma\rvert)$	$\mathcal{O}(N + \lvert\Sigma\rvert)$	Verified
Q7	Minimise Array Deviation	Two-Way Greedy Monotonic Halving	$\mathcal{O}(n \log(M) \log n)$	$\mathcal{O}(n)$	Verified
Q8	Meeting Rooms Allocation	Interval Coloring / Min-Heap of Ends	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q9	Hu-Tucker Tree Simulation	3-Phase Compatible Greedy Alphabetic Tree	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Verified
Q10	Greedy Superstring Open	SCS Heuristic & Sept 2026 AI Disproof	$\mathcal{O}(n^3 L)$	$\mathcal{O}(nL)$	Explored

Q1: Fractional Knapsack with Deterioration Rate

Exchange Priority · $\mathcal{O}(n \log n)$

Problem Statement: n items with base values v_i, weights w_i, decay rates $\lambda_i > 0$. At time t, value density decays to $\left(\frac{v_i}{w_i}\right) - \lambda_i t$. Knapsack capacity is W. Determine optimal scheduling order and fractional choices to maximize total value.

Mathematical Formulation & Exchange Argument: Consuming weight w_i' over $[t_s, t_s + w_i']$ at rate 1 yields: $V_{i(t_s,w_i')} = \int_{t_s}^{t_s+w_i'} \left(\frac{v_i}{w_i} - \lambda_i \tau\right) d\tau = w_i' \left[\frac{v_i}{w_i} - \lambda_i \left(t_s + \frac{w_i'}{2}\right)\right]$. Comparing adjacent schedules $A \to B$ vs $B \to A$: $V(A \to B) - V(B \to A) = w_A w_B (\lambda_A - \lambda_B)$. Since weights $w_A, w_B > 0$, items with **higher decay rates λ_i must strictly precede** items with lower decay rates to minimize deterioration losses. Ties are broken by descending base density $\frac{v_i}{w_i}$.

Economic Cutoff Invariant: Marginal density must be strictly positive: $t_{\max}(i) = \frac{v_i}{w_i \lambda_i}$. If $t_s \geq t_{\max}(i)$, the item is omitted to avoid value destruction.

Complexity: Time: $\mathcal{O}(n \log n)$ (QuickSort), Auxiliary Space: $\mathcal{O}(n)$.

Validation Trace ($W = 50$):

Item	v, w	λ	Interval	Weight	Val Gained
2	$100, 20$	0.8	$[0, 6.25]$	6.25	15.6250
1	$60, 10$	0.5	$[6.25, 12]$	5.75	8.2656
3	$120, 30$	0.2	$[12, 20]$	8.00	6.4000

Total Weight Consumed: $\frac{20.00}{50.00}$, **Maximum Total Value:** 30.2906.

```c
typedef struct {
    int id; double v, w, lambda, density, fraction;
    double valGained, startTime, finishTime;
} Item;

int compareItems(const void *a, const void *b) {
    const Item *ia = (const Item *)a, *ib = (const Item
*)b;
    if (ib->lambda > ia->lambda) return 1;
    if (ib->lambda < ia->lambda) return -1;
    return (ib->density > ia->density) ? 1 : -1;
}

/* Fractional Greedy Scheduler with Zero-Density Cutoff
*/
for (int i = 0; i < n; i++) {
    if (curWeight >= W) break;
    double rem = W - curWeight;
    double take = (items[i].w < rem) ? items[i].w : rem;
    double tMax = items[i].density / items[i].lambda;
    if (curTime >= tMax) continue;
    if (curTime + take > tMax) take = tMax - curTime;
    double avgD = items[i].density -
                  items[i].lambda * (curTime + take /
2.0);
    double val = take * avgD;
    if (val > 0.0) {
        items[i].fraction = take / items[i].w;
        items[i].valGained = val;
        items[i].startTime = curTime;
        items[i].finishTime = curTime + take;
        curTime += take; curWeight += take; totalVal +=
val;
    }
}
```

Q2: Canonical Huffman Coding

Prefix Trees & Length-Ordered Bitstrings · $\mathcal{O}(n \log n)$

Problem Statement: Given symbol frequencies, construct a prefix-free binary code of minimum expected length, and output the canonical Huffman codebook (where codes are ordered lexicographically by length).

Greedy Choice & Information Theoretic Bounds: Huffman extracts two minimal frequency nodes from a Min-Heap, guaranteeing deepest sibling optimality (CLRS Lemma 16.2 & 16.3). Kraft-McMillan condition: $\sum_{i=1}^{n} 2^{-L_i} \leq 1$. Shannon Entropy lower bound: $H(\Sigma) = -\sum p_i \log_2 p_i \leq L(C) < H(\Sigma) + 1$.

Canonical Recurrence: Standard trees require transmitting topology trees. Canonical codes standardize bitstrings purely by length array:
1. Sort symbols by length L_i ascending, breaking ties by symbol ASCII.
2. Set $C_0 = 0$. For $i > 0$: $C_i = (C_{i-1} + 1) \ll (L_i - L_{i-1})$.

Complexity: Time: $\mathcal{O}(n \log n)$ (min-heap + `qsort`), Space: $\mathcal{O}(n)$.

CLRS 16 Benchmark Trace ($N = 100$):

Sym	Freq	Prob	Length	Canonical Code
'A'	45	0.45	1	0
'B'	13	0.13	3	100
'C'	12	0.12	3	101
'D'	16	0.16	3	110
'E'	9	0.09	4	1110
'F'	5	0.05	4	1111

Entropy: 2.2199, **Avg Length:** 2.2400, **Coding Efficiency:** 99.10%.

```
Node *buildHuffmanTree(char *symbols, int *freqs, int n)
{
    MinHeap *heap = createMinHeap(n);
    for (int i = 0; i < n; i++)
        heap->array[i] = createNode(symbols[i],
freqs[i]);
    heap->size = n;
    for (int i = (n - 1) / 2; i >= 0; i--)
minHeapify(heap, i);
    while (heap->size > 1) {
        Node *left = extractMin(heap);
        Node *right = extractMin(heap);
        Node *top = createNode('$', left->freq + right-
>freq);
        top->left = left; top->right = right;
        insertMinHeap(heap, top);
    }
    return extractMin(heap);
}
void buildCanonicalCodes(SymbolCode *codes, int n) {
    qsort(codes, n, sizeof(SymbolCode),
compareCanonical);
    unsigned int cur = 0; int curLen = codes[0].length;
    for (int i = 0; i < n; i++) {
        if (codes[i].length > curLen) {
            cur <<= (codes[i].length - curLen);
            curLen = codes[i].length;
        }
        codes[i].canonicalCode = cur++;
    }
}
```

Q3: Minimum Initial Fuel (Reverse Greedy)

Retroactive Max-Heap Simulation · $\mathcal{O}(n \log n)$

Problem Statement: A vehicle needs to reach target D starting with fuel F. Along the way are stations (d_i, f_i) (distance from origin and fuel stock). Find the minimum number of refueling stops required.

Retroactive / Reverse Greedy Choice Property: The vehicle drives monotonically forward consuming 1 unit fuel per unit distance.
1. Continue forward until fuel is exhausted before next waypoint d_i.
2. Greedily refuel retroactively from the passed station offering **maximum fuel stock** using a Max-Heap.
3. Maximizing added fuel expands the reachable horizon maximally.
4. If the heap is empty during a deficit, destination is unreachable (-1).

Complexity: Time: $\mathcal{O}(n \log n)$ (sorting + heap ops), Auxiliary Space: $\mathcal{O}(n)$.

Highway Telemetry ($D = 100, F = 10$): Stations: $(10, 60), (20, 30), (30, 30), (60, 40)$.
- At $x = 10$: Fuel deficit! Refuels at Station 1 ($+60L \to 60$).
- At $x = 60$: Fuel $10L$, needs $40L$ to target. Refuels at Station 4 ($+40L \to 50$).
- Reaches target $D = 100$ with $10L$ remaining.

Minimum Refueling Stops: 2 (Stations 1 and 4).

```
int minRefuelStops(int D, int F, Station *stns, int n) {
    qsort(stns, n, sizeof(Station), compareStations);
    stns[n].id = -1; stns[n].distance = D; stns[n].fuel
= 0;
    MaxHeap *heap = createMaxHeap(n + 5);
    int curFuel = F, curPos = 0, stops = 0;
    for (int i = 0; i <= n; i++) {
        int dist = stns[i].distance - curPos;
        while (curFuel < dist) {
            if (heap->size == 0) return -1; /*
Unreachable */
            HeapElement best = extractMax(heap);
            curFuel += best.station.fuel;
            stops++;
        }
        curFuel -= dist;
        curPos = stns[i].distance;
        if (i < n) {
            HeapElement elem = {stns[i], stns[i].fuel};
            insertMaxHeap(heap, elem);
        }
    }
    return stops;
}
```

Q4: Minimum Cost to Connect Sticks

Huffman Binary Merge Tree Equivalence · $\mathcal{O}(n \log n)$

Problem Statement: Connecting two sticks of lengths x and y costs $x + y$, leaving a stick of length $x + y$. Find the minimum total connection cost to connect all n sticks into one single stick.

Huffman Merge Tree Isomorphism: Connecting n sticks constructs a full binary merge tree T. Original sticks reside as leaves. Every leaf L_i contributes its length to all $d_{T(i)}$ ancestor merges: $\text{Total Cost} = \sum_{u \in \text{ Internal}(T)} W(u) = \sum_{i=1}^{n} L_i \cdot d_{T(i)}$. This cost function is isomorphic to the external path length of Huffman trees!

Greedy Choice: Greedily extracting the two shortest sticks from a Min-Heap minimizes external path length. 64-bit integer (`long long`) prevents overflow on large stick lengths.

Complexity: Time: $\mathcal{O}(n \log n)$ (Floyd build $\mathcal{O}(n) + n - 1$ extractions), Space: $\mathcal{O}(n)$.

Step-by-Step Ledger (Sticks $[1, 8, 3, 5]$):

Step	Stick A	Stick B	Merge Cost	Cumulative
1	1	3	4	4
2	4	5	9	13
3	8	9	17	30

Final Combined Length: 17, **Minimum Total Cost: 30**.

```
long long connectSticks(long long *sticks, int n) {
    if (n <= 1) return 0;
    MinHeap *heap = createMinHeap(n);
    for (int i = 0; i < n; i++) heap->array[i] =
sticks[i];
    heap->size = n;
    for (int i = (n - 1) / 2; i >= 0; i--)
minHeapify(heap, i);
    long long totalCost = 0;
    while (heap->size > 1) {
        long long first = extractMin(heap);
        long long second = extractMin(heap);
        long long mergeCost = first + second;
        totalCost += mergeCost;
        insertMinHeap(heap, mergeCost);
    }
    return totalCost;
}
```

Q5: Candy Distribution Problem

Bi-directional Slope Greedy Sweep · $\Theta(n)$

Problem Statement: n children stand in line with ratings $r_0, ..., r_{n-1}$. Each child gets ≥ 1 candy. Children rated strictly higher than immediate neighbors must receive more candies than them. Minimize total candies.

Slope Decoupling & Component-wise Optimality: A single pass fails because descending slopes create arbitrary right-to-left dependencies ($10 > 8 > 5 > 2$).

1. **Pass 1 (Left-to-Right):** $c_i = c_{i-1} + 1$ if $r_i > r_{i-1}$ else 1. (Satisfies left neighbor condition).
2. **Pass 2 (Right-to-Left):** $c_i = \max(c_i, c_{i+1} + 1)$ if $r_i > r_{i+1}$. (Satisfies right neighbor condition while preserving left).

Taking $\max$ preserves monotonic upward slopes from both directions. The assignment $c_i = \max(c_i^L, c_i^R)$ is component-wise minimal, guaranteeing minimal sum $\sum c_i$.

Complexity: Time: $\Theta(n)$ (two linear scans), Auxiliary Space: $\Theta(n)$.

Verification Traces:

- Peak $[1, 0, 2] \rightarrow$ Forward: $[1, 1, 2] \rightarrow$ Backward: $\{2, 1, 2\}$. Total: **5**.
- Descent $[12, 8, 5, 2, 1] \rightarrow$ Backward: $\{5, 4, 3, 2, 1\}$. Total: **15**.
- Equal Ratings $[1, 2, 2] \rightarrow$ Neighbor condition is non-strict: $\{1, 2, 1\}$. Total: **4**.

```
int *candyDistribution(int n, int *ratings) {
    int *candies = (int *)calloc(n, sizeof(int));
    for (int i = 0; i < n; i++) candies[i] = 1;
    /* Pass 1: Forward Sweep (Left-to-Right) */
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1])
            candies[i] = candies[i - 1] + 1;
    }
    /* Pass 2: Backward Sweep (Right-to-Left) */
    for (int i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1]) {
            if (candies[i] < candies[i + 1] + 1)
                candies[i] = candies[i + 1] + 1;
        }
    }
    return candies;
}
```

Q6: Reorganise String with K-Distance Apart

Max-Heap & Sliding Cooldown Queue · $\mathcal{O}(N \log|\Sigma|)$

Problem Statement: Given string S and integer K, rearrange S such that identical characters are separated by at least distance K ($j - i \geq K$). If impossible, return an empty string.

Pigeonhole Feasibility & Cooldown Barrier: Placing $f_{\max}$ copies of the dominant character requires: $N \geq (f_{\max} - 1)K + m$.

Greedy Character Placement: At each slot i, extract the eligible character with highest remaining frequency from a Max-Heap (most constrained first). Placed characters enter a circular FIFO cooldown queue of size K with release time $i + K$. If heap empties while unplaced characters remain, deadlock proves rearrangement is impossible ("").

Complexity: Time: $\mathcal{O}(N \log|\Sigma|) = \mathcal{O}(N)$ ($|\Sigma| \leq 256$), Space: $\mathcal{O}(N + |\Sigma| + K)$.

Validation Traces:
- $S = \text{aabbcc}, K = 3 \rightarrow$ Result: acbacb (Verified valid, distance between identical characters is 3).
- $S = \text{aaabc}, K = 3 \rightarrow$ Dominant `'a'` ($f = 3$) requires $7 > 5$ slots $\Rightarrow$ Returns "" (Impossible).

```
char *reorganizeStringK(char *inputStr, int k) {
    int n = strlen(inputStr);
    if (k <= 1 || n <= 1) return strdup(inputStr);
    int count[256] = {0};
    for (int i = 0; i < n; i++) count[(unsigned
char)inputStr[i]]++;
    MaxHeap *heap = createMaxHeap(256);
    for (int i = 0; i < 256; i++)
        if (count[i] > 0) insertMaxHeap(heap, (CharFreq)
{(char)i, count[i]});
    char *res = (char *)malloc(n + 1);
    Queue *cd = createQueue(k + 5);
    for (int i = 0; i < n; i++) {
        while (!isQueueEmpty(cd) && cd->array[cd-
>front].releaseTime <= i)
            insertMaxHeap(heap, dequeue(cd).item);
        if (heap->size == 0) return ""; /* Deadlock */
        CharFreq b = extractMax(heap); res[i] = b.ch;
        if (--b.freq > 0) enqueue(cd, (CooldownItem){b,
i + k});
    }
    res[n] = '\0'; return res;
}
```

Q7: Minimise Deviation in Array

Two-Way Greedy Monotonic Halving · $\mathcal{O}(n \log M \log n)$

Problem Statement: Positive integer array A. Operations: multiply odd by 2, or divide even by 2. Minimize deviation $(\max(A) - \min(A))$ after any number of operations.

Monotonic Reduction to One-Way Greedy: Odd numbers can only double once; even numbers can only halve.
1. Pre-double every odd element: $x \rightarrow 2x$.
2. All elements are now instantiated at their theoretical ceiling.
3. **Directional Invariant:** No element can ever increase again; the only legal operation is division by 2!
4. In a Max-Heap, repeatedly pop maximum M. Update deviation $\Delta = \min(\Delta, M - \text{minVal})$. If M is even, push $\frac{M}{2}$ and update minVal. If M is odd, it cannot decrease; search terminates at global optimum.

Complexity: Time: $\mathcal{O}(n \log(M) \log n)$, Space: $\mathcal{O}(n)$.

Descent Trace ($[4, 1, 5, 20, 3]$): Odds doubled: $[4, 2, 10, 20, 6]$ (Min 2, Max 20, Dev 18).
- Halve $20 \rightarrow 10$ (Dev 8) $\rightarrow$ Halve $10 \rightarrow 5$ (Dev 4) $\rightarrow$ Halve $6 \rightarrow 3$ (Dev 3).
- Top is 5 (odd). Halving stopped. **Min Deviation: 3**.

```
int minimumDeviation(int *arr, int n) {
    MaxHeap *heap = createMaxHeap(n + 5);
    int minVal = INT_MAX;
    for (int i = 0; i < n; i++) {
        int v = (arr[i] % 2 != 0) ? arr[i] * 2 : arr[i];
        heap->array[i] = v;
        if (v < minVal) minVal = v;
    }
    heap->size = n;
    for (int i = (n - 1) / 2; i >= 0; i--)
maxHeapify(heap, i);
    int minDev = heap->array[0] - minVal;
    while (heap->size > 0) {
        int m = extractMax(heap);
        if (m - minVal < minDev) minDev = m - minVal;
        if (m % 2 == 0) {
            int half = m / 2;
            if (half < minVal) minVal = half;
            insertMaxHeap(heap, half);
        } else break; /* Odd max cannot be halved */
    }
    return minDev;
}
```

Q8: Minimum Number of Meeting Rooms

Interval Graph Coloring & Earliest-Start Scan · $\mathcal{O}(n \log n)$

Problem Statement: Given meeting intervals $[[s_1, e_1], [s_2, e_2], ...]$, find the minimum number of conference rooms required.

Interval Perfection & Maximum Clique: In interval conflict graphs, chromatic number equals maximum clique depth: $\chi(G) = \omega(G) = d = \max_t |\{i : s_i \leq t < e_i\}|$.

Earliest-Start Priority Queue: Sort meetings by start time s_i. Track active room departure times in a Min-Heap. If $e_{\min} \leq s_i$, reuse the room; otherwise open a new room. Kleinberg & Tardos Theorem 4.4 proves room count never exceeds d.

Complexity: Time: $\mathcal{O}(n \log n)$ (qsort + heap), Space: $\mathcal{O}(n)$.

Trace ($[[0, 30], [5, 10], [15, 20]]$):
1. $[0, 30] \rightarrow$ Opens Room 1 ($e = 30$).
2. $[5, 10] \rightarrow 30 > 5$, Opens Room 2 ($e = 10$).
3. $[15, 20] \rightarrow 10 \leq 15$, Reuses Room 2 ($e = 20$).

Min Rooms Needed: 2 (Room 1: M1; Room 2: M2 → M3).

```
int minMeetingRooms(Meeting *meetings, int n) {
    qsort(meetings, n, sizeof(Meeting), compareStart);
    MinHeap *heap = createMinHeap(n + 5);
    int rooms = 0;
    for (int i = 0; i < n; i++) {
        if (heap->size > 0 && heap->array[0].endTime <=
meetings[i].start) {
            Room r = extractMinRoom(heap);
            r.endTime = meetings[i].end;
            insertMinHeap(heap, r);
        } else {
            Room nr = {++rooms, meetings[i].end};
            insertMinHeap(heap, nr);
        }
    }
    return rooms;
}
```

Q9: Hu-Tucker Greedy Simulation

Optimal Alphabetic Binary Search Tree · $\mathcal{O}(n^2)$

Problem Statement: Given ordered sequence weights $w_1, ..., w_n$, construct an optimal alphabetic binary search tree minimizing $\sum w_i \cdot \text{depth}(i)$ while preserving strict in-order sequence of leaves.

Alphabetic Trees vs. Huffman: Standard Huffman merges any two elements regardless of order. Hu-Tucker preserves in-order through 3 greedy phases:
1. **Combination:** In working sequence, merge compatible pair (u, v) with min weight sum. Two nodes are compatible iff **no unmerged leaves exist between them**. Replace u with composite node; delete v.
2. **Level Assignment:** Extract leaf depths d_i from Phase 1 tree (Hu-Tucker theorem proves these depths are optimal).
3. **Reconstruction:** Pair adjacent nodes at max level $L_{\max}$ bottom-up to build planar in-order tree.

Complexity: Time: $\mathcal{O}(n^2)$ (or $\mathcal{O}(n \log n)$ with heaps), Space: $\mathcal{O}(n)$.

Five-Key Trace ($[10, 2, 3, 1, 5]$): Levels: $d = [1, 3, 4, 4, 2]$. Cost: $10(1) + 2(3) + 3(4) + 1(4) + 5(2) = \mathbf{42}$. Unrestricted Huffman: 41. Alphabetic Preservation Penalty: $+1$ (2.44%).

```
void huTuckerCombine(HTNode **seq, int *seqLen) {
    while (*seqLen > 1) {
        int bI = -1, bJ = -1; long long minSum = -1;
        for (int i = 0; i < *seqLen - 1; i++) {
            for (int j = i + 1; j < *seqLen; j++) {
                int comp = 1;
                for (int k = i + 1; k < j; k++)
                    if (seq[k]->isLeaf) { comp = 0;
break; }
                if (comp) {
                    long long s = seq[i]->weight +
seq[j]->weight;
                    if (minSum == -1 || s < minSum)
                        { minSum = s; bI = i; bJ = j; }
                } else break;
            }
        }
        HTNode *p = createInternal(minSum, seq[bI],
seq[bJ]);
        seq[bI] = p;
        for (int k = bJ; k < *seqLen - 1; k++) seq[k] =
seq[k + 1];
        (*seqLen)--;
    }
}
```

Q10: Greedy Superstring Conjecture (Open Problem & September 2026 Disproof) *SCS Overlap Digraph & Modern Disproof* · $\mathcal{O}(n^3 L)$

Problem Statement: Given string set $\mathcal{S} = \{s_1, ..., s_n\}$, find the shortest single string containing every s_i. Explore the 40-year-old 2-approximation conjecture and the September 2026 ArXiv disproof paper.

SCS Complexity & Overlap Heuristic: Shortest Common Superstring is NP-hard and APX-hard. Dual to finding the Maximum-Weight Hamiltonian Path in the overlap digraph. The classic greedy heuristic repeatedly merges the pair (u, v) with maximum suffix-to-prefix overlap $\text{ov}(u, v)$.

The 40-Year Conjecture & Sept 2026 Disproof: Hypothesized $|T_{\text{greedy}}| \leq 2.0 |T_{\text{opt}}|$ (Gallant et al. 1980). A September 2026 ArXiv paper proved that for even string length $k \geq 10$: $R(k) \geq \frac{9k+2}{4k+4} \to \frac{9}{4} = \mathbf{2.25}$ **as** $k \to \infty$. This conclusively disproves the 2-approximation conjecture! Notably, the counterexample family was found with AI reasoning assistance.

Verification Trace: {"CATG", "ATGC", "TGCA", "GCAT"}: Merges with overlap 3 producing "CATGCAT" (Len 7, $\text{OPT} = 7$, Ratio 1.000).

```
char *greedySuperstring(int n, char **strings) {
    int count = n;
    char **act = cloneStrings(strings, n);
    while (count > 1) {
        int bU = -1, bV = -1, maxOv = -1;
        for (int i = 0; i < count; i++)
            for (int j = 0; j < count; j++)
                if (i != j && computeOverlap(act[i],
act[j]) > maxOv)
                    { maxOv = computeOverlap(act[i],
act[j]); bU = i; bV = j; }
        char *m = mergeStrings(act[bU], act[bV], maxOv);
        free(act[bU]); free(act[bV]); act[bU] = m;
        for (int k = bV; k < count - 1; k++) act[k] =
act[k + 1];
        count--;
    }
    return act[0];
}
```

Algorithmic Synthesis & Cross-Problem Audit

Q. No.	Problem Name	Key Theoretical Invariant	Core Data Structure	Efficiency
Q1	Fractional Knapsack	Decay exchange priority (λ_i) ordering	Sorted Array + Cutoff	$\mathcal{O}(n \log n)$
Q2	Canonical Huffman	Sibling optimality at maximum depth	Min-Heap + Bitwise Shifts	$\mathcal{O}(n \log n)$
Q3	Refueling Stops	Retroactive maximal fuel replenishment	Max-Heap (Reverse Greedy)	$\mathcal{O}(n \log n)$
Q4	Connect Sticks	Huffman merge tree external path length	Min-Heap (64-bit int)	$\mathcal{O}(n \log n)$
Q5	Candy Distribution	Bi-directional slope constraint decoupling	Two Linear Pass Arrays	$\Theta(n)$
Q6	Reorganise String	Most-constrained character priority + barrier	Max-Heap + FIFO Queue	$\mathcal{O}(N \log \lvert\Sigma\rvert)$
Q7	Array Deviation	Monotonic descent from upper bound ceiling	Max-Heap Priority Queue	$\mathcal{O}(n \log M \log n)$
Q8	Meeting Rooms	Interval chromatic number $\chi(G) = \omega(G) = d$	Min-Heap of End Times	$\mathcal{O}(n \log n)$
Q9	Hu-Tucker Trees	Compatible pair merge preserves optimal depth	Working Sequence List	$\mathcal{O}(n^2)$
Q10	Superstring Open	Max-overlap Hamiltonian path & 2026 disproof	Overlap Digraph / Greedy	$\mathcal{O}(n^3 L)$

Pedagogical Takeaways:

1. **Exchange Arguments & Matroids:** Proved in Q1, Q3, Q4; swapping an arbitrary optimal solution's choice with the greedy choice maintains feasibility without degrading cost.
2. **State-Space Monotonicity (Q7):** Pre-doubling odds collapses bi-directional search into a monotonic one-way greedy descent.
3. **Limits of Greedy Heuristics (Q10):** Local maximum overlap choices can create pathological traps, demonstrating why approximation bounds require formal proof and can be disproved (2.25).

Q. No.	Target Metric	Status
Q1–Q4	Exact Optimal	PASS
Q5–Q8	Optimal Bound	PASS
Q9–Q10	Tree / Superstr	PASS

STUDENT CERTIFICATION: All mathematical formulations, exchange proofs, C programs, and traces represent my verified academic work.

Signature: Rishav Shaw (Roll: B125100, CSE-B1)

FACULTY EVALUATION: Verified algorithm correctness, exchange proofs, priority queue invariants, and execution traces for Lab 09.

Instructor Signature: Dr. Ajaya Kumar Dash